



T E M A 4

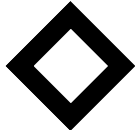
# ANÁLISIS LÉXICO

Longart, Daniela. V-31.445.710.  
Martinez, Fabiana. V-30.498.516.  
Ortiz, Sebastian. V-30.576.350.  
Valencia, Haddan. V-31.818.222.

# ¿QUE ES UN LEXER?

Es la primera fase de un compilador. Transforma caracteres de un programa fuente en unidades con significado lógico.

*Conceptos Clave: Un Token es la categoría lógica, mientras que el Lexema es el valor real del código que coincide con ese Token. Utilizamos Flex, un metacompilador que automatiza la generación de este analizador.*



# METODOLOGÍA Y FUNCIONAMIENTO DE FLEX

*Para construir este componente, en lugar de programar autómatas y condicionales a mano desde cero, utilizamos Flex. El flujo de trabajo técnico que seguimos consta de tres pasos críticos:*



## GENERACIÓN

Pasamos nuestro archivo `lexer.l` por Flex, el cual traduce las expresiones regulares a una matriz de estados de un Autómata Finito Determinista y nos devuelve un archivo en C estándar llamado



## COMPILACIÓN

Utilizamos el compilador GCC (bajo el entorno MSYS2) para compilar este archivo C y generar un binario ejecutable nativo.



## EJECUCIÓN

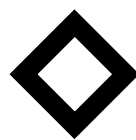
El binario final expone una función interna llamada `yylex()`. Cuando se ejecuta, esta función abre el archivo fuente de prueba, lee su buffer y empieza a segmentar los tokens







# DISEÑO DEL LENGUAJE L Y EL CÓDIGO FUENTE



El archivo de Flex que construimos se divide estrictamente en las tres secciones estándar del metacompilador:



## EN LA SECCIÓN DE DEFINICIONES

incluimos las cabeceras de entrada y salida estándar de C y configuramos la opción `noyywrap`, lo que le indica al escáner que solo procesará un único archivo fuente por ejecución.

## EN LA SECCIÓN DE REGLAS

emparejamos los patrones mediante expresiones regulares con sus respectivas acciones. Por ejemplo, definimos palabras reservadas estrictas de Rust como `fn`, `let`, `mut`, `if`, `else` y `return`.

## EN LA SECCIÓN DE CÓDIGO DE USUARIO

implementamos la función `main()`. Esta recibe la ruta del archivo de código de Rust por línea de comandos, redirige el puntero del archivo al flujo de entrada de Flex mediante la variable global `yylin`, e invoca a `yylex()` para iniciar el escaneo



# DISEÑO DEL LENGUAJE L Y EL CÓDIGO FUENTE

El archivo de Flex que construimos se divide estrictamente en las tres secciones estándar del metacompilador:

## EN LA SECCIÓN DE DEFINICIONES

incluimos las cabeceras de entrada y salida estándar de C y configuramos la opción `noyywrap`, lo que le indica al escáner que solo procesará un único archivo fuente por ejecución.

```
Analisis Lexico > Lex.l
5  %option noyywrap
6
7  %%
8
9  "fn"      { printf("TOKEN: KW_FN, Lexema: %s\n", yytext); }
10 "let"     { printf("TOKEN: KW_LET, Lexema: %s\n", yytext); }
11 "mut"     { printf("TOKEN: KW_MUT, Lexema: %s\n", yytext); }
12 "if"      { printf("TOKEN: KW_IF, Lexema: %s\n", yytext); }
13 "else"    { printf("TOKEN: KW_ELSE, Lexema: %s\n", yytext); }
14 "return"  { printf("TOKEN: KW_RETURN, Lexema: %s\n", yytext); }
15
16 "i32"|"f64"|"bool" { printf("TOKEN: TIPO_DATO, Lexema: %s\n", yytext); }
17
18 [a-zA-Z_][a-zA-Z0-9_]* { printf("TOKEN: IDENTIFICADOR, Lexema: %s\n", yytext); }
19
20 [0-9]+ "." [0-9]+ { printf("TOKEN: NUM_DECIMAL, Lexema: %s\n", yytext); }
21 [0-9]+          { printf("TOKEN: NUM_ENTERO, Lexema: %s\n", yytext); }
22
23 "="           { printf("TOKEN: OP_ASIGNACION, Lexema: %s\n", yytext); }
24 "+"|"-"|"*"|"|" { printf("TOKEN: OP_ARITMETICO, Lexema: %s\n", yytext); }
25 "=="|"!="|">"|"<"|>="|<=" { printf("TOKEN: OP_RELACIONAL, Lexema: %s\n", yytext); }
26
27 "{"|"}"|"("|")"|";"|":"|"," { printf("TOKEN: SIMBOLO, Lexema: %s\n", yytext); }
28
29 [ \t\n\r]+ { /* Ignorar espacios, tabulaciones y saltos de línea */ }
30 "/*".* { /* Ignorar comentarios de una línea */ }
31
32 .          { printf("ERROR LEXICO: Caracter no reconocido: %s\n", yytext); }
33
34 %%
```

# DISEÑO DEL LENGUAJE L Y EL CÓDIGO FUENTE

El archivo de Flex que construimos se divide estrictamente en las tres secciones estándar del metacompilador:

## EN LA SECCIÓN DE REGLAS

emparejamos los patrones mediante expresiones regulares con sus respectivas acciones. Por ejemplo, definimos palabras reservadas estrictas de Rust como fn, let, mut, if, else y return.

```
Analisis Lexico > Lex.l
5  %option noyywrap
6
7  %%
8
9  "fn"      { printf("TOKEN: KW_FN, Lexema: %s\n", yytext); }
10 "let"     { printf("TOKEN: KW_LET, Lexema: %s\n", yytext); }
11 "mut"     { printf("TOKEN: KW_MUT, Lexema: %s\n", yytext); }
12 "if"      { printf("TOKEN: KW_IF, Lexema: %s\n", yytext); }
13 "else"    { printf("TOKEN: KW_ELSE, Lexema: %s\n", yytext); }
14 "return"  { printf("TOKEN: KW_RETURN, Lexema: %s\n", yytext); }
15
16 "i32"|"f64"|"bool" { printf("TOKEN: TIPO_DATO, Lexema: %s\n", yytext); }
17
18 [a-zA-Z_][a-zA-Z0-9_]* { printf("TOKEN: IDENTIFICADOR, Lexema: %s\n", yytext); }
19
20 [0-9]+ "." [0-9]+ { printf("TOKEN: NUM_DECIMAL, Lexema: %s\n", yytext); }
21 [0-9]+          { printf("TOKEN: NUM_ENTERO, Lexema: %s\n", yytext); }
22
23 "="            { printf("TOKEN: OP_ASIGNACION, Lexema: %s\n", yytext); }
24 "+"|"-"|"*"|"|" { printf("TOKEN: OP_ARITMETICO, Lexema: %s\n", yytext); }
25 "=="|"!="|">"|"<"|>="|<=" { printf("TOKEN: OP_RELACIONAL, Lexema: %s\n", yytext); }
26
27 "{"|"}"|"("|")"|";"|":"|"," { printf("TOKEN: SIMBOLO, Lexema: %s\n", yytext); }
28
29 [ \t\n\r]+ { /* Ignorar espacios, tabulaciones y saltos de línea */ }
30 "//".* { /* Ignorar comentarios de una línea */ }
31
32 .          { printf("ERROR LEXICO: Caracter no reconocido: %s\n", yytext); }
33
34 %%
```

# DISEÑO DEL LENGUAJE L Y EL CÓDIGO FUENTE

El archivo de Flex que construimos se divide estrictamente en las tres secciones estándar del metacompilador:

## EN LA SECCIÓN DE CÓDIGO DE USUARIO

implementamos la función `main()`. Esta recibe la ruta del archivo de código de Rust por línea de comandos, redirige el puntero del archivo al flujo de entrada de Flex mediante la variable global `yyin`, e invoca a `yylex()` para iniciar el escaneo

```
36 int main(int argc, char **argv) {
37     if (argc > 1) {
38         FILE *file = fopen(argv[1], "r");
39         if (!file) {
40             perror("Error al abrir el archivo");
41             return 1;
42         }
43         yyin = file;
44     } else {
45         printf("Por favor, indique el archivo de entrada. Ejemplo: ./Lex1 codigo.rs\n");
46         return 1;
47     }
48
49     yylex();
50     return 0;
51 }
```

# RESULTADOS

Esta actividad nos demuestra que el uso de metacompiladores como Flex es el estándar industrial en la construcción de compiladores modernos. Reducen drásticamente las líneas de código manuales, garantizan una velocidad de procesamiento óptima gracias a la generación de AFDs eficientes en C y nos permiten enfocarnos puramente en el diseño de la gramática y las reglas del lenguaje. Muchas gracias por su atención, quedo atento a sus preguntas

```
M /c/Users/Daniela Longart/Documents/Universidad/Lenguajes y compiladores/Análisis Lexico
Daniela Longart@DESKTOP-OKRQN72 UCRT64 /c/Users/Daniela Longart/Documents/Universidad/Lenguajes y compiladores/Análisis Lexico
$ ./lexer_rust codigo.rs
-bash: ./lexer_rust: No such file or directory

Daniela Longart@DESKTOP-OKRQN72 UCRT64 /c/Users/Daniela Longart/Documents/Universidad/Lenguajes y compiladores/Análisis Lexico
$ ./Lex1_rust codigo.rs
TOKEN: KW_FN, Lexema: fn
TOKEN: IDENTIFICADOR, Lexema: calcular_suma
TOKEN: SIMBOLO, Lexema: (
TOKEN: IDENTIFICADOR, Lexema: a
TOKEN: SIMBOLO, Lexema: :
TOKEN: TIPO_DATO, Lexema: i32
TOKEN: SIMBOLO, Lexema: ,
TOKEN: IDENTIFICADOR, Lexema: b
TOKEN: SIMBOLO, Lexema: :
TOKEN: TIPO_DATO, Lexema: i32
TOKEN: SIMBOLO, Lexema: )
TOKEN: SIMBOLO, Lexema: {
TOKEN: KW_LET, Lexema: let
TOKEN: KW_MUT, Lexema: mut
TOKEN: IDENTIFICADOR, Lexema: resultado
TOKEN: OP_ASIGNACION, Lexema: =
TOKEN: IDENTIFICADOR, Lexema: a
TOKEN: OP_ARITMETICO, Lexema: +
TOKEN: IDENTIFICADOR, Lexema: b
TOKEN: SIMBOLO, Lexema: ;
TOKEN: KW_IF, Lexema: if
TOKEN: IDENTIFICADOR, Lexema: resultado
TOKEN: OP_RELACIONAL, Lexema: >
TOKEN: NUM_ENTERO, Lexema: 10
TOKEN: SIMBOLO, Lexema: {
TOKEN: KW_RETURN, Lexema: return
TOKEN: IDENTIFICADOR, Lexema: resultado
TOKEN: SIMBOLO, Lexema: ;
TOKEN: SIMBOLO, Lexema: }
TOKEN: SIMBOLO, Lexema: }
```